Figure 8.5 When event handlers that are subscribed to a Kafka topic are triggered, the event is produced. The event message must be in the right format as it needs to be converted to the `Movie` object for inserting into the database.

If you want to run the Kafka service alongside the tests to see how the actual Kafka implementation works, then read on or skip to section 8.4 as this information is optional. Ensure you have Docker installed, as stated in the requirements in section 8.2.1. Start the Kafka cluster with Docker Compose by running the script from the command line:

```
npm run start:kafka
```

Launch the docker-compose file, which contains the Kafka instance, Kafka UI, and `init` script, to set up the "movies" topic. A snippet showing the core components is provided in the following listing.

Listing 8.7 Docker Compose Kafka cluster

```
version: '3.8'

services:
  kafka:
    image: confluentinc/cp-kafka:6.0.14
    platform: linux/amd64
    depends_on:
      - zookeeper
    ports:
      - '29092:29092'
      ...

  init-kafka:
    image: confluentinc/cp-kafka:6.0.14
    depends_on:
      - kafka
  ...

  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    ports:
      - 8085:8080
  ...

  zookeeper:
    image: confluentinc/cp-zookeeper:6.0.14
    platform: linux/amd64
    ports:
      - '22181:2181'
  ...
```

The Kafka UI is available on port 8085, which shows the local Kafka cluster running with the "movies" topic available, as shown in figure 8.6.